🎯 Project Title: Employee Attendance and Payroll Tracker

🏢 Background & Real-World Scenario

ATH Pvt. Ltd. is a growing IT services company with multiple departments such as Software Development, Quality Assurance, Human Resources, Finance, and Technical Support. Each department consists of employees with different job roles, work shifts, and pay structures. Currently, ATH maintains attendance and salary records manually using spreadsheets — a process that has led to frequent payroll errors, mismatched attendance entries, and delays in salary disbursement.

To overcome these inefficiencies, the management has decided to develop a **centralized Employee Attendance and Payroll Tracker System**. The system should not only record attendance but also automatically calculate salaries, generate performance-related insights, and maintain transparent payroll records that can be accessed by HR, Finance, and departmental heads.

🧠 Project Overview

The **Employee Attendance and Payroll Tracker** aims to build a complete automation system that combines attendance management, employee tracking, and payroll computation using **Python (for logic and interface)** and **SQL (for database management)**. Students must conceptualize, design, and implement this system in a way that ensures **accuracy, scalability, and real-time accessibility**.

The system must handle various real-world complexities such as:

- Multiple work shifts (Day, Night, Rotational).
- Employees joining mid-month or leaving before salary date.
- Half-days, work-from-home status, and official leaves.
- D.B. → Monthly payroll reports with department-wise breakdown.
- Tax and deduction adjustments. ← Python
- HR dashboards for attendance and performance patterns. ← CSV, PowerBI

Students should analyze and define the logical structure to support these cases using relational concepts — **Primary Keys, Foreign Keys, Joins, Grouping, and Aggregation**.

System Requirements & Scenarios

1. Employee Management Scenario

Each employee has personal, departmental, and financial details.

Python →

- HR should be able to add, update, or deactivate an employee.
- When a new employee joins, their base salary, joining date, and assigned department should be recorded.
- When an employee resigns, the system should generate a *pro-rated salary* till the last working day. → Python

2. Attendance Recording Scenario

Employees can mark attendance via digital entry (simulated through Python input or data import).

Python menu →

- Attendance statuses: *Present, Absent, Half Day, Work from Home, Official Leave.*
- Late arrivals beyond a certain threshold (e.g., 10:30 AM) should trigger an automatic deduction in salary or flag for HR review.
- Attendance should be stored day-wise, and reports should be able to summarize monthly presence, absence, and late records per employee.

3. Payroll Calculation Scenario

Payroll must automatically calculate based on attendance data:

- Full-day: full salary for that day.
- Half-day: 50% salary.
- Absent: 0 salary.
- Work from home: considered full-day, but tracked separately.
- For every employee, total working days × per-day rate = base salary.
- Apply deductions (late marks, absences) and allowances (performance bonus, overtime pay).
- Final salary should include tax and provident fund deductions.

Students must implement **Python logic** to automate these calculations, while using **SQL joins and aggregations** to fetch consolidated data.

4. Department & Role Management Scenario

The organization has multiple departments, and every employee is assigned to one.

- Generate reports like total payroll per department, total working days, or average attendance rate.

- Identify departments with maximum overtime or absenteeism using `GROUP BY` and `HAVING` clauses.

5. Analytics & Insights Scenario

The HR manager wants to identify:

- Employees with attendance below 75%.
- Departments where total payroll cost exceeded a certain limit.
- Top 5 employees with the least absenteeism or highest overtime.
- Monthly report summarizing salary distribution and attendance trends.

These queries require students to **join multiple tables** and use **aggregation functions** to extract meaningful data.

6. System Operations via Python

Students must create a **menu-driven Python application** that interacts with the database:

- Add / Update / Delete employee details
- Mark daily attendance (accept multiple inputs at once)
- Generate salary slips for each employee (text/console output)
- Produce department-wise payroll summaries
- Export reports (CSV or text)
- Handle invalid inputs and database exceptions gracefully

The application should show how **Python executes SQL commands (CRUD)** dynamically and fetches complex joined data.

⚙️ Project Challenges

This project intentionally introduces real-life difficulties to encourage critical thinking:

- Handling missing attendance records (what if an employee forgets to mark attendance?).
- Managing salary calculations for employees who joined mid-month.
- Detecting incorrect data entries or mismatched employee-department assignments.
- Optimizing SQL queries to avoid redundancy when generating reports.
- Designing a scalable database structure that can support multiple branches in the future.

Students are expected to document their **design decisions**, explaining how their structure prevents data duplication, ensures referential integrity, and supports analytical reporting.

📋 Expected Deliverables

1. **Comprehensive Case Study Report**
   - Problem definition & objectives
   - Functional and non-functional requirements
   - Entity identification and relationship mapping (ER Diagram)
   - System workflow diagram
   - Detailed explanation of payroll and attendance logic
2. **SQL Scripts**
   - Database creation, sample data, complex queries for analysis
3. **Python Application**
   - Menu-based interface performing CRUD and payroll generation
   - Well-structured code with exception handling and clear output formatting
4. **Output Demonstrations**
   - Screenshots of Python console outputs
   - Example reports generated through SQL/Python integration

---

📃 Evaluation Criteria

Component	Description	Weight
Database design and query logic	Proper normalization, use of joins, and aggregation queries	30%
Python integration and automation	Functional, menu-driven interface with proper CRUD and reporting	30%
Documentation and analysis	Depth of case study, clarity, and completeness of explanation	25%
Creativity and handling of edge cases	Smart solutions for practical challenges	15%